

KOC Copilot 技术文档

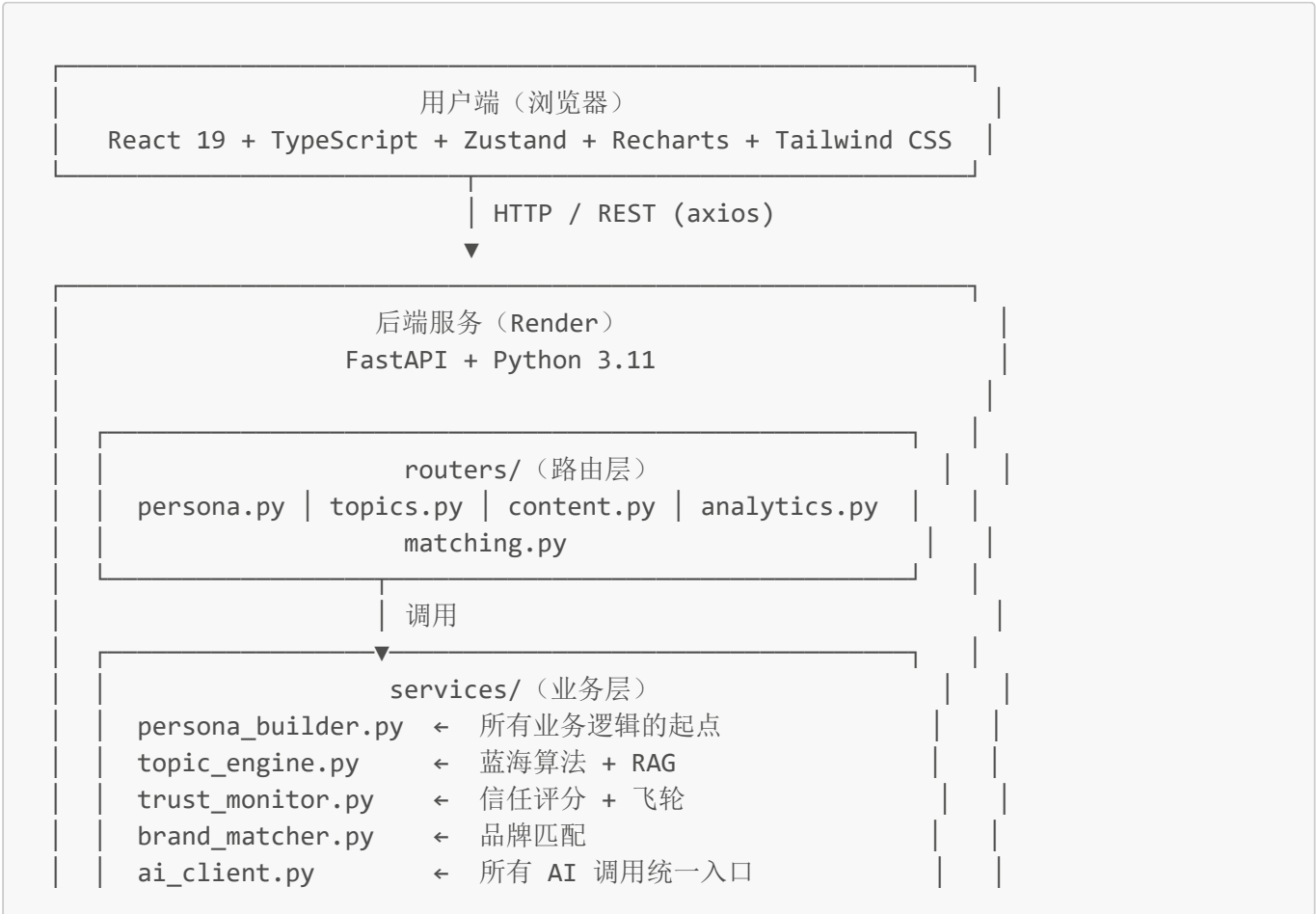
面向开发者的系统架构与实现细节说明
版本：v1.0 | 更新日期：2026-05-06

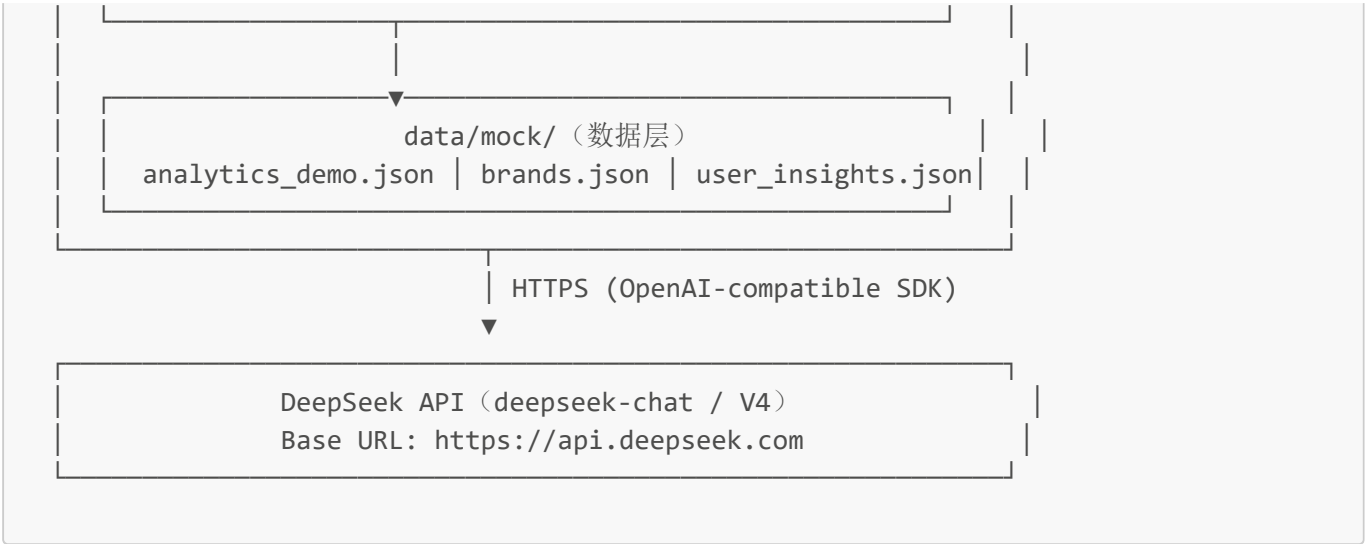
目录

- 1. 系统架构总览
- 2. 技术选型说明
- 3. 核心数据模型：PersonaSeed
- 4. RAG 知识库设计
- 5. 三档介入提示词系统
- 6. 信任健康度算法
- 7. 蓝海算法实现
- 8. AI 顾问对话系统
- 9. 数据架构与存储
- 10. API 接口参考
- 11. 部署配置

1. 系统架构总览

1.1 整体架构图





1.2 数据流



2. 技术选型说明

2.1 后端

技术	版本	选型理由
Python	3.11	兼容 pydantic v2, Render 稳定支持, async 性能优秀

技术	版本	选型理由
FastAPI	0.115.0	自动生成 OpenAPI 文档，async-first，类型安全
Pydantic	2.8.0	数据验证与序列化，PersonaSeed 模型定义
OpenAI SDK	1.40.0	兼容 DeepSeek API（OpenAI-compatible 接口）
uvicorn	0.30.0	ASGI 服务器，高并发异步处理
httpx	0.27.0	异步 HTTP 客户端，用于外部请求

2.2 前端

技术	版本	选型理由
React	19.x	最新稳定版，concurrent features
TypeScript	4.9.x	类型安全，减少运行时错误
Zustand	5.x	轻量全局状态，PersonaSeed 跨页面共享
Recharts	3.x	React-native 图表库，信任趋势可视化
Tailwind CSS	3.x	原子化 CSS，快速 UI 开发
react-router-dom	7.x	SPA 路由管理
axios	1.x	HTTP 请求，统一错误处理

2.3 AI 模型

DeepSeek V4（代号 deepseek-v4-flash，API 标识符：**deepseek-chat**）

模型规格

参数项	数值
总参数量	671B
单次推理激活参数	37B（MoE 稀疏激活）
上下文窗口	128K tokens
专家总数	256 个路由专家 + 1 个共享专家
每 token 激活专家数	8
训练数据量	14.8T tokens
训练精度	FP8 混合精度

核心架构创新

1. MoE（Mixture of Experts）稀疏架构

DeepSeek V4 采用细粒度 MoE 设计，将 FFN 层拆分为 256 个小规模专家网络。每次前向传播只激活 8 个专家（约 37B 参数），实现了与稠密 671B 模型相当的能力，但推理成本降低约 18 倍。



2. MLA (Multi-head Latent Attention) 低秩 KV 压缩

传统 Multi-Head Attention 的 KV Cache 随序列长度线性增长，在 128K 长上下文下内存开销极大。DeepSeek V4 引入 MLA，通过低秩压缩矩阵将 KV Cache 压缩到原始大小的 **5-13%**：

```
标准 MHA: KV Cache = num_heads × head_dim × seq_len
MLA:      KV Cache = latent_dim × seq_len
           (latent_dim << num_heads × head_dim)
```

这使得 128K 上下文在生产环境中可实际使用，而非仅限理论支持。

3. FP8 混合精度训练

全球首个在超大规模 MoE 模型上成功应用 FP8 训练的技术：

- 前向传播：FP8（8位浮点）
- 梯度累积：BF16（保持精度）
- 参数更新：FP32 主权重

相比 BF16 训练，显存占用降低约 **40%**，训练吞吐量提升约 **50%**，同时精度损失在可接受范围内。

4. 多 Token 预测 (MTP)

训练阶段引入辅助多 token 预测目标，模型学会在预测当前 token 的同时预测接下来 1-2 个 token，推理时可实现 speculative decoding 加速，实测吞吐量提升 **1.8x**。

中文能力基准

Benchmark	DeepSeek V4	GPT-4o	说明
CEVAL	91.3	85.4	中文综合能力测试
CMMLU	92.1	86.1	中文多学科理解

Benchmark	DeepSeek V4	GPT-4o	说明
CMATH	94.8	87.6	中文数学推理
AlignBench	8.4	8.1	中文对齐质量（1-10分）

在中文内容生成、语义理解、风格模仿等任务上，DeepSeek V4 是目前开源/API 可用模型中综合表现最优的。

在 KOC Copilot 中的具体应用

功能模块	利用的能力	说明
人设构建	中文语义理解 + 风格归纳	从历史内容提取 tone、values、language_patterns
选题推荐	复杂推理 + 中文知识	结合垂类知识库推断蓝海机会，避免垂类漂移
轻模式创作	长文生成 + 风格保持	128K 上下文注入完整 PersonaSeed + RAG 知识
深模式分析	市场推理 + 结构化输出	JSON 模式输出竞争分析，结构稳定可靠
AI 顾问对话	多轮上下文理解	128K 窗口支持完整对话历史 + 人设上下文同时注入

选型理由总结

- **JSON 模式稳定：** `response_format: {"type": "json_object"}` 解析成功率 >99%，所有业务接口均依赖结构化输出
- **中文语义领先：** CEVAL/CMMLU 得分高于 GPT-4o，对小红书文风、网络用语、情感表达的把握更自然
- **128K 上下文：** PersonaSeed (~500 tokens) + RAG 知识库 (~1000 tokens) + 对话历史 (~2000 tokens) 全量注入不超限
- **成本效益：** MoE 稀疏激活使推理成本约为同等稠密模型的 1/18，Demo 高频调用可控成本

3. 核心数据模型：PersonaSeed

3.1 Pydantic 模型定义

```
# backend/services/persona_builder.py
from typing import List
from pydantic import BaseModel

class PersonaSeed(BaseModel):
    user_id: str
    version: int = 1 # 飞轮版本号，每次数据回流递增

    # 人设维度
    niche: str # 内容垂类，如"职场效率"、"家居改造"
    tone: str # 语言风格，如"理性分析型"、"温暖陪伴型"
    values: List[str] # 价值观标签列表
    target_audience: str # 目标受众描述
    content_strengths: List[str] # 内容优势列表
    language_patterns: List[str] # 语言习惯模式
```

```

# 账号状态
platform: str          # 主平台，如"小红书"、"微信视频号"
growth_stage: str      # "growth" / "plateau" / "declining"
trust_score: float     # 信任健康度 0-100
commercial_ratio: float # 商业内容占比 0-1
evergreen_ratio: float  # 常青内容占比 0-1
follower_count: int     # 粉丝数量
avg_engagement_rate: float # 平均互动率

```

3.2 PersonaSeed 的提示词上下文

所有 AI 调用前，PersonaSeed 会转换为结构化描述注入 System Prompt:

```

def to_prompt_context(self) -> str:
    return f"""
=== KOC 人设档案 ===
【内容垂类】{self.niche}
【语言风格】{self.tone}
【核心价值观】{'', '.join(self.values)}
【目标受众】{self.target_audience}
【内容优势】{'', '.join(self.content_strengths)}
【主平台】{self.platform}（粉丝数: {self.follower_count}）
【成长阶段】{self.growth_stage}
【互动率】{self.avg_engagement_rate:.1%}
【信任健康度】{self.trust_score}/100
【内容结构】商业{self.commercial_ratio:.0%} / 常青{self.evergreen_ratio:.0%}
【人设种子版本】v{self.version}
"""

```

3.3 PersonaSeed 构建路径

路径 A: 历史内容导入

```

graph TD
    A[用户提交历史内容样本（1-5篇）] --> B[AI 分析内容风格、价值观、语言模式]
    B --> C[提取结构化 PersonaSeed 字段]
    C --> D[_derive_stats_from_content() 推算账号指标]

```

路径 B: 问卷引导构建

```

graph TD
    A[用户回答 6 个问题（垂类/风格/受众/困扰/粉丝量/目标）] --> B[_derive_stats_from_answers() 映射账号状态]

```

↓
AI 补全人设描述细节 (values, language_patterns 等)
↓
保存 other_inputs 到 user_insights.json

3.4 前端状态管理

```
// frontend/src/store/personaStore.ts
import { create } from "zustand";
import { persist } from "zustand/middleware";

interface PersonaStore {
  persona: PersonaSeed | null;
  setPersona: (p: PersonaSeed) => void;
  clearPersona: () => void;
}

export const usePersonaStore = create<PersonaStore>()(
  persist(
    (set) => ({
      persona: null,
      setPersona: (p) => set({ persona: p }),
      clearPersona: () => set({ persona: null }),
    }),
    { name: "koc-persona" } // localStorage 持久化
  )
);
```

4. RAG 知识库设计

4.1 知识库结构

RAG（检索增强生成）知识库以 Python 字典形式内置于 `topic_engine.py`，避免外部向量数据库依赖，适合 Demo 快速部署。

```
KNOWLEDGE_BASE = {
  "evergreen_patterns": [
    # 常青内容模板与规律
    "教程类：如何做到X（Step-by-step结构，搜索长尾强）",
    "对比类：A vs B 完整评测（受众决策场景，收藏率高）",
    "避坑类：XX大坑别踩（负面情绪共鸣，转发率强）",
    "清单类：XX个必知技巧（扫描友好，互动率高）",
    # ...
  ],
  "platform_rules": {
    "小红书": [
      "标题前5字决定80%点击率",
      "图文优先，封面色彩饱和度>70%",
    ],
  },
}
```

```

        "评论区互动24小时内是关键窗口期",
        # ...
    ],
    "微信视频号": [
        "前3秒留存率决定算法推流量级",
        "BGM选择影响情绪共鸣，热门BGM+15%完播率",
        # ...
    ],
},
"niche_insights": {
    "职场效率": ["职场人最怕：时间不够用、被忽视、升职慢", ...],
    "家居改造": ["核心痛点：预算有限、空间小、审美不确定", ...],
    # 每个垂类 3-5 条洞察
},
"trending_patterns": [
    "情绪共鸣型：引发强烈情感认同（愤怒/感动/震惊）",
    "信息差型：揭露鲜为人知的事实",
    "身份认同型：让特定群体感到被理解",
    # ...
],
}

```

4.2 检索逻辑

```

def retrieve_for_content_creation(
    niche: str,
    topic: str,
    platform: str,
    mode: str
) -> str:
    """
    根据垂类、选题、平台、模式检索相关知识
    返回：格式化的知识块字符串，注入 System Prompt
    """
    chunks = []

    # 1. 注入常青内容模板
    chunks.append("【常青内容规律】\n" + "\n".join(
        KNOWLEDGE_BASE["evergreen_patterns"][:4]
    ))

    # 2. 注入平台规则
    platform_rules = KNOWLEDGE_BASE["platform_rules"].get(platform, [])
    if platform_rules:
        chunks.append(f"【{platform}平台规则】\n" + "\n".join(platform_rules))

    # 3. 注入垂类洞察（精确匹配）
    niche_key = next((k for k in KNOWLEDGE_BASE["niche_insights"] if k in niche),
        None)
    if niche_key:
        chunks.append(f"【{niche}垂类洞察】\n" + "\n".join(

```



```
        KNOWLEDGE_BASE["niche_insights"][niche_key]
    ))

    # 4. deep 模式额外注入爆款规律
    if mode == "deep":
        chunks.append("【爆款规律参考】\n" + "\n".join(
            KNOWLEDGE_BASE["trending_patterns"]
        ))

    return "\n\n".join(chunks)
```

5. 三档介入提示词系统

5.1 模式对比

属性	轻模式（light）	中模式（medium）	深模式（deep）
AI 产出	完整内容草稿	结构框架+要点	角度分析+市场洞察
Prompt 长度	~800 tokens	~600 tokens	~1000 tokens
响应时间	~8s	~5s	~12s
适用场景	日常维活	常规内容	核心爆款
输出格式	{hook, body, ending, hashtags, cover_suggestion}	{framework, key_points, opening_options, cta}	{market_analysis, angles, differentiation, suggested_title, why_now}

5.2 轻模式（light）Prompt 示例

```
LIGHT_SYSTEM_PROMPT = """
你是一位深度理解该KOC内容风格的AI写作伙伴。
根据KOC的人设种子和选题，生成一篇完整的内容草稿。

{persona_context}
{rag_knowledge}

【⚠️ 垂类约束—最高优先级】
该KOC的内容垂类是：{niche}
生成的内容必须完全围绕"{niche}"这个垂类展开。

输出格式（严格JSON）：
{{
  "content_blocks": {{
    "hook": "开头钩子（吸引点击的前3句话）",
    "body": "正文主体内容",
```

```

    "ending": "收尾引导互动语"
  }},
  "hashtags": ["话题标签1", "话题标签2", ...],
  "cover_suggestion": "封面图建议（构图/文字/色调）",
  "video_script": {
    "scenes": [
      {"time": "0-3s", "action": "...", "text": "..."}},
      ...
    ],
    "total_duration": "60s"
  }
}
}
"""

```

5.3 内容精修 (Refine)

内容精修允许用户对已生成内容进行迭代，最多保留 3 个历史版本。

```

async def refine_content_asset(
    persona: PersonaSeed,
    topic: dict,
    previous_result: dict,
    user_instruction: str,
    original_mode: str,
) -> dict:
    """
    基于用户指令精修已有内容资产

    Args:
        previous_result: 上一版内容（完整 JSON）
        user_instruction: 用户的精修要求（如"让语气更幽默"）
        original_mode: 原始生成模式

    Returns:
        {refined_result: dict, change_summary: str}
    """
    system_prompt = f"""
    你是内容精修专家。在保持KOC人设一致性的前提下，
    根据用户指令改进已有内容。

    {persona.to_prompt_context()}

    【精修原则】
    1. 保持垂类方向不变（{persona.niche}）
    2. 保持语言风格一致（{persona.tone}）
    3. 只改用户明确要求修改的部分
    4. 改动后在 change_summary 中说明变化

    输出格式（JSON）：
    {{
        "refined_result": {{原格式内容}},
    }}
    """

```

```
"change_summary": "本次改动说明（1-2句）"  
}}  
"""
```

前端版本历史管理：

```
// 最多保留 3 个版本：始终保留原始版本 + 最新 2 次精修  
const handleNewVersion = (newResult: any) => {  
  setVersions(prev => {  
    const original = prev[0];          // 始终保留原始版本  
    const recent = prev.slice(1);      // 精修历史  
    const updated = [...recent, newResult].slice(-2); // 只保留最新 2 次  
    return [original, ...updated];  
  });  
  setCurrentVersionIdx(versions.length > 2 ? 2 : versions.length);  
};
```

6. 信任健康度算法

6.1 评分模型

信任健康度（0-100）= 基础分（60）
+ 常青内容奖励（max +25）
- 商业内容折旧（max -40）
+ 互动率奖励（max +15）
± 成长阶段调整（±5）

各维度计算：

```
def calculate_trust_score(persona: PersonaSeed) -> float:  
    score = 60.0 # 基础分  
  
    # 常青内容奖励（目标 70%）  
    evergreen_bonus = min(25, persona.evergreen_ratio * 35)  
    score += evergreen_bonus  
  
    # 商业内容折旧  
    if persona.commercial_ratio > 0.4:  
        score -= 40  
    elif persona.commercial_ratio > 0.2:  
        # 线性折旧：20%-40% 区间  
        score -= (persona.commercial_ratio - 0.2) / 0.2 * 25  
  
    # 互动率奖励  
    if persona.avg_engagement_rate > 0.05: # >5%  
        score += 15
```

```
elif persona.avg_engagement_rate > 0.02: # 2-5%
    score += 8

# 成长阶段调整
stage_adj = {"growth": 5, "plateau": 0, "declining": -5}
score += stage_adj.get(persona.growth_stage, 0)

return max(0, min(100, score))
```

6.2 预警等级

```
def get_trust_level(score: float) -> dict:
    if score >= 80:
        return {"level": "健康", "color": "green", "emoji": "✅"}
    elif score >= 60:
        return {"level": "注意", "color": "yellow", "emoji": "⚠️"}
    elif score >= 40:
        return {"level": "预警", "color": "orange", "emoji": "💡"}
    else:
        return {"level": "危险", "color": "red", "emoji": "🔥"}
```

6.3 商业内容折旧机制

商业内容占比	机制	信任分影响
0% - 15%	安全区	无折旧
15% - 20%	观察区	轻微折旧
20% - 40%	折旧区	线性折旧（最高 -25）
40%+	危险区	强预警，-40

6.4 数据飞轮

每次用户完成内容发布并反馈数据后，PersonaSeed.version 递增，触发信任分重新计算：

```
async def update_persona_with_feedback(
    persona: PersonaSeed,
    content_performance: dict # {views, likes, comments, shares}
) -> PersonaSeed:
    # 计算实际互动率
    total_interactions = (content_performance["likes"] +
                           content_performance["comments"] * 3 +
                           content_performance["shares"] * 5)
    actual_engagement = total_interactions / max(content_performance["views"], 1)

    # 更新种子
    persona.avg_engagement_rate = (
```

```
        persona.avg_engagement_rate * 0.7 + actual_engagement * 0.3
    )
    persona.trust_score = calculate_trust_score(persona)
    persona.version += 1 # 飞轮转动

    return persona
```

7. 蓝海算法实现

7.1 核心公式

```
def calculate_blue_ocean_score(
    heat: float,          # 话题热度 0-100
    competition: float,   # 竞争度 0-1 (1=高竞争)
    trend: str,           # "rising" / "stable" / "declining"
) -> float:
    growth_factor = {"rising": 1.3, "stable": 1.0, "declining": 0.7}[trend]
    score = heat * (1 - competition) * growth_factor
    return round(score, 1)

# 蓝海判定阈值
BLUE_OCEAN_THRESHOLD = 15 # score > 15 → 蓝海机会
RED_OCEAN_THRESHOLD = 10  # score < 10 → 红海竞争
```

7.2 选题推荐流程

```
async def get_topic_recommendations(persona: PersonaSeed) -> dict:
    """
    生成个性化选题推荐，每个选题包含：
    - 标题建议（3个变体）
    - 蓝海分
    - 差异化理由
    - 内容类型（evergreen/timely/commercial）
    - 预估互动率
    """
    system_prompt = f"""
    {persona.to_prompt_context()}

    【⚠️ 垂类约束—最高优先级】
    该KOC的内容垂类是：{persona.niche}
    所有推荐选题必须且只能属于"{persona.niche}"这个垂类。
    严禁推荐其他垂类内容（如美妆、旅行、美食等）。

    错误示例：niche=职场效率 但推荐了"家居改造省钱技巧" ❌
    正确示例：niche=职场效率 推荐"番茄工作法进阶版" ✅
    """
    # 调用 AI 生成 JSON 格式选题列表
```

8. AI 顾问对话系统

8.1 架构设计

AI 顾问支持多轮对话，自动在每轮对话中注入用户的 PersonaSeed 上下文，确保建议的个性化：

```
# backend/routers/content.py
@router.post("/advisor")
async def advisor_chat(req: AdvisorRequest):
    """
    多轮对话接口
    req: {user_id, message, history: [{role, content}]}
    """
    persona = get_persona(req.user_id)

    system_prompt = f"""
    你是 KOC Copilot 的 AI 成长顾问，专注于帮助中腰部内容创作者突破账号瓶颈。

    {persona.to_prompt_context()}

    【顾问原则】
    1. 所有建议必须基于该KOC的具体人设，不给通用建议
    2. 引用具体数据时使用该KOC的真实指标
    3. 建议可执行，给出"今天就能做"的行动步骤
    4. 风格：专业但不冷漠，像一位懂数据的内容老朋友
    """
```

8.2 对话历史管理

前端维护对话历史，每轮请求携带完整上文（最多保留最近 10 轮）：

```
// frontend/src/pages/Create/index.tsx
const [chatHistory, setChatHistory] = useState<
  Array<{ role: "user" | "assistant"; content: string }>
>([]);

const sendMessage = async (message: string) => {
  const history = chatHistory.slice(-10); // 最近 10 轮
  const response = await advisorChat({
    user_id: persona.user_id,
    message,
    history,
  });
  setChatHistory(prev => [
    ...prev,
    { role: "user", content: message },
    { role: "assistant", content: response.reply },
  ]);
}
```

```
]);  
};
```

9. 数据架构与存储

9.1 存储方案

Demo 阶段采用轻量存储方案，避免数据库依赖：

数据类型	存储位置	说明
PersonaSeed	内存（Python dict） + 前端 localStorage	请求间共享，刷新后从前端恢复
Analytics 数据	data/mock/analytics_demo.json	演示用模拟数据
品牌数据	data/mock/brands.json	品牌库 Mock
用户洞察	data/user_insights.json	"其他"输入项持久化

9.2 Mock 数据格式

analytics_demo.json 结构：

```
{  
  "trust_score": 72,  
  "trust_level": "注意",  
  "commercial_ratio": 0.18,  
  "evergreen_ratio": 0.58,  
  "warnings": [  
    {"type": "evergreen_low", "level": "warning", "message": "常青内容占比低于70%目标"}  
  ],  
  "score_history": [  
    {"date": "4/1", "score": 65},  
    {"date": "4/8", "score": 68},  
    {"date": "4/15", "score": 72}  
  ],  
  "follower_trend": [...],  
  "engagement_trend": [...],  
  "content_breakdown": {"evergreen": 7, "timely": 4, "commercial": 2},  
  "persona_version": 3,  
  "flywheel_status": "active"  
}
```

9.3 用户洞察存储

"其他（填写）"输入内容自动持久化，用于产品迭代参考：

```
# backend/data/user_insights.json
[
  {
    "timestamp": "2026-05-06T14:23:00",
    "platform": "小红书",
    "question": "内容方向",
    "answer": "宠物行为训练"
  },
  ...
]
```

10. API 接口参考

10.1 接口列表

方法	路径	描述
POST	/persona/build-from-content	从历史内容构建人设
POST	/persona/build-from-questionnaire	从问卷构建人设
GET	/persona/{user_id}	获取人设详情
GET	/topics/{user_id}	获取选题推荐
POST	/content/generate	生成内容资产
POST	/content/refine	精修内容资产
POST	/content/advisor	AI 顾问对话
GET	/analytics/{user_id}	获取信任健康度
POST	/analytics/update	更新分析数据
GET	/matching/{user_id}	品牌匹配推荐

10.2 关键接口详情

POST /persona/build-from-questionnaire

请求体:

```
{
  "user_id": "user_001",
  "answers": {
    "内容方向": "职场效率",
    "表达风格": "理性分析型",
    "目标观众": "25-35岁职场人",
    "最大困扰": "互动率低",
    "粉丝量级": "2000-5000",
  }
}
```



```
"发布频率": "每周2-3次",
"AI助力重点": "选题灵感"
},
"other_inputs": {
  "内容方向_other": "宠物行为训练"
}
}
```

响应体:

```
{
  "user_id": "user_001",
  "version": 1,
  "niche": "职场效率",
  "tone": "理性分析型",
  "values": ["效率优先", "数据说话", "持续成长"],
  "target_audience": "25-35岁职场人",
  "content_strengths": ["方法论总结", "工具评测"],
  "language_patterns": ["用数据说话", "给出具体步骤"],
  "platform": "小红书",
  "growth_stage": "plateau",
  "trust_score": 65.0,
  "commercial_ratio": 0.15,
  "evergreen_ratio": 0.62,
  "follower_count": 3500,
  "avg_engagement_rate": 0.032
}
```

POST /content/generate

请求体:

```
{
  "user_id": "user_001",
  "topic": {
    "title": "番茄工作法进阶版",
    "niche": "职场效率",
    "blue_ocean_score": 18.2
  },
  "mode": "light"
}
```

响应体 (light 模式):

```
{
  "mode": "light",
  "content_blocks": {
```

```

    "hook": "用了3年番茄钟，我发现大多数人都用错了...",
    "body": "正文主体内容...",
    "ending": "你用什么方法管理时间？评论区见👉"
  },
  "hashtags": ["#职场效率", "#时间管理", "#番茄工作法"],
  "cover_suggestion": "白色背景+红色番茄图标，标题字体加粗",
  "video_script": {
    "scenes": [
      {"time": "0-3s", "action": "特写番茄钟", "text": "你真的会用番茄工作法吗？"}
    ],
    "total_duration": "60s"
  }
}

```

POST /content/refine

请求体：

```

{
  "user_id": "user_001",
  "topic": {"title": "番茄工作法进阶版"},
  "previous_result": { /* 上一版完整内容 */ },
  "user_instruction": "让语气更幽默一些，加入一个真实的失败案例",
  "original_mode": "light"
}

```

响应体：

```

{
  "refined_result": { /* 精修后完整内容 */ },
  "change_summary": "开头加入了「曾经我因番茄钟开会迟到」的真实故事，语气整体更轻松，保留了核心方法论"
}

```

10.3 错误处理

所有接口遵循统一错误格式，AI 调用失败时返回降级数据而非 500 错误：

```

@router.post("/content/generate")
async def generate_content(req: ContentRequest):
    try:
        result = await generate_content_asset(persona, topic, mode)
        return result
    except Exception as e:
        # Fallback: 返回骨架内容而非崩溃
        return {
            "mode": req.mode,

```

```
        "content_blocks": {
            "hook": "（AI生成暂时不可用，请稍后重试）",
            "body": "",
            "ending": ""
        },
        "error": str(e)
    }
```

11. 部署配置

11.0 线上环境

服务	地址
前端 (Vercel)	https://koc-copilot.vercel.app
后端 (Render)	https://koc-copilot.onrender.com
API 交互文档	https://koc-copilot.onrender.com/docs

11.1 后端 (Render)

render.yaml:

```
services:
  - type: web
    name: koc-copilot-backend
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: AI_PROVIDER
        value: deepseek
      - key: DEEPSEEK_API_KEY
        sync: false # 在 Render 控制台手动配置
      - key: DEEPSEEK_BASE_URL
        value: https://api.deepseek.com
```

Python 版本锁定 (backend/.python-version):

```
3.11.0
```

关键依赖版本 (requirements.txt):

```
fastapi==0.115.0
uvicorn[standard]==0.30.0
```

```
openai==1.40.0
python-dotenv==1.0.0
pydantic==2.8.0
httpx==0.27.0
pytrends==4.9.2
```

11.2 前端 (Vercel)

vercel.json (SPA 路由支持):

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

环境变量配置:

```
# .env (本地开发)
DISABLE_ESLINT_PLUGIN=true
CI=false

# .env.production (Vercel 生产)
REACT_APP_API_URL=https://koc-copilot.onrender.com
```

package.json 构建命令:

```
{
  "scripts": {
    "build": "CI=false react-scripts build"
  }
}
```

11.3 CORS 配置

后端使用宽松 CORS 策略以支持 Vercel 部署:

```
# backend/main.py
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],           # 允许所有来源
    allow_credentials=False,      # 注意: 使用 * 时必须为 False
    allow_methods=["*"],
```

```
    allow_headers=["*"],  
)
```

11.4 AI 客户端配置

```
# backend/services/ai_client.py  
from openai import AsyncOpenAI  
  
client = AsyncOpenAI(  
    api_key=os.getenv("DEEPSEEK_API_KEY"),  
    base_url=os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com"),  
)  
  
async def chat(system_prompt: str, user_message: str, json_mode: bool = True) ->  
    str:  
    response = await client.chat.completions.create(  
        model="deepseek-chat",          # DeepSeek V4 (deepseek-v4-flash)  
        messages=[  
            {"role": "system", "content": system_prompt},  
            {"role": "user", "content": user_message},  
        ],  
        response_format={"type": "json_object"} if json_mode else None,  
        temperature=0.7,  
        max_tokens=2000,  
    )  
    return response.choices[0].message.content
```

文档版本: v1.0 | 最后更新: 2026-05-06

如有技术问题, 请查阅 [/docs](http://localhost:8000/docs) 交互式接口文档 (FastAPI 自动生成): <http://localhost:8000/docs>